DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROJECT SPECIFICATION & ARCHITECTURE REPORT

Edu Track – Student Academic Management System

A Robust, Role-Governed Full-Stack University Operating Suite with Real-Time Timetable Conflict Engines, Anti-Tamper Attendance, Section-Scoped Coursework, and Automated Proctoring

Project Title:	Edu Track Academic Management System
Architecture Pattern:	Multi-Tier MERN Architecture + Dual-Store High-Availability Engine
Client Technology:	React 18, TypeScript, Vite, Tailwind CSS, Lucide Icons, Recharts
Server Technology:	Node.js, Express, Socket.IO, JWT Security, Multer
Persistence Layer:	MongoDB Atlas Cloud Database + Autonomous Local JSON Store Engine
Deployment Topology:	Vercel Global Edge Network (Frontend) + Dedicated Node Service (Backend)
Security Model:	Role-Based Access Control (Admin, Faculty, Student) with Stateless JWT
Document Release:	Production Engineering Version 2.4.0 (November 2026)

Abstract: *Higher education institutions face mounting challenges when operating across disconnected software utilities for course scheduling, daily attendance verification, learning asset distribution, and student examination proctoring. Edu Track establishes a singular, integrated, role-governed platform uniting Institutional Administrators, Teaching Faculty, and Enrolled Students into an event-driven operating environment. Built upon modern web primitives, the system introduces: (1) an automated scheduling conflict detection engine preventing room, instructor, and student cohort overlaps; (2) an anti-tamper single-day attendance logging workflow; (3) strict section-level isolation for coursework and study materials; (4) an online examination suite featuring tab-switch proctoring and immediate auto-evaluation; and (5) an autonomous dual-store persistence engine ensuring zero operational interruption if cloud databases become unreachable. This document presents the definitive Software Requirements Specification (SRS), system architecture, detailed algorithms, and test validations of the platform.*

1. Introduction & Project Scope

Modern university governance requires rigorous record-keeping, conflict-free instructional scheduling, dependable attendance metrics, and transparent academic evaluation pipelines. Traditional setups typically rely on disparate legacy software packages or paper-driven rosters, leading to administrative bottlenecks, scheduling clashes, data synchronization latency, and security vulnerabilities.

1.1 Core Objectives

- **Unified Academic Operating Environment:** Eliminate operational silos by centralizing user management, timetable planning, attendance records, coursework sharing, examinations, and grade point calculations.
- **Proactive Conflict Prevention:** Validate multi-dimensional constraints (classroom capacity, instructor allocation, student cohort schedule) at time of timetable entry rather than discovering clashes in production.
- **Continuous High Availability:** Guarantee uninterrupted campus workflows through an autonomous dual-store engine that falls back to local structured JSON persistence if cloud databases drop offline.
- **Integrity in Daily Attendance:** Lock session marking strictly to the active calendar date, preventing backdated attendance fraud and unverified historic record updates.
- **Fair & Monitored Online Testing:** Provide web-based assessments with strict time-window enforcement, automatic tab-switch detection, automated objective evaluation, and faculty-governed score releases.

1.2 System Stakeholders & User Classes

The platform defines three strictly segregated user classes with distinct operational responsibilities:

- **Institutional Administrator:** Full governance over academic departments, course subjects, instructional cohorts, student rosters, faculty appointments, semester examination schedules, and institutional audit trails.
- **Teaching Faculty:** Confined strictly to assigned instructional sections and courses. Marks daily student attendance, uploads section-scoped study materials, publishes coursework, authors online quizzes, and releases scores.
- **Enrolled Student:** Accesses personalized weekly schedules, department semester exam dates, personal attendance percentages, section study materials, online assessments, and official cumulative academic transcripts.

2. Software Requirements Specification (SRS)

2.1 Functional Requirements Matrix

Req ID	Module	Functional Specification	Priority
FR-01	Authentication	Stateless JWT authentication with encrypted credential verification and role claim validation.	**CRITICAL**
FR-02	Administration	Creation and persistent storage of academic departments, subjects, student and faculty accounts.	**CRITICAL**
FR-03	Class Timetable	Weekly instructional schedule matrix with automated multi-variable collision detection engine.	**HIGH**
FR-04	Exam Timetable	Schedule semester exams with department-filtered subject dropdown and universal student visibility.	**HIGH**
FR-05	Attendance	Lock session marking to current calendar date; provide binary Present/Absent batch toggles.	**HIGH**
FR-06	Section Scoping	Restrict faculty visibility strictly to allocated sections and assigned departmental courses.	**CRITICAL**
FR-07	Study Materials	Upload instructional documents with section-scoped distribution and student download access.	**MEDIUM**
FR-08	Coursework Tasks	Publish assignments; capture student file/text submissions in a faculty review ledger.	**MEDIUM**
FR-09	Online Exams	Create time-bounded MCQ assessments with passing cutoffs, timer, and tab-switch proctoring.	**HIGH**
FR-10	Auto-Grading	Deterministically evaluate student answers against correct keys upon submission.	**HIGH**
FR-11	Result Release	Faculty-governed release trigger unlocking student scorecards and review breakdown.	**HIGH**
FR-12	Dual-Store Sync	Seamless failover between MongoDB Atlas and atomic JSON persistence engine.	**CRITICAL**

2.2 Non-Functional Requirements (NFR)

- **NFR-01 Performance:** REST API response latency under 60 milliseconds for cached and index-backed read operations.
- **NFR-02 High Availability:** 99.95% system availability maintained through autonomous dual-store failover engine.
- **NFR-03 Data Integrity:** Complete foreign key referential consistency across rosters, timetables, and exam submissions.
- **NFR-04 Security & Privacy:** Passwords hashed with bcrypt (salt rounds = 10); strict section-level isolation prevents cross-cohort data leakage.
- **NFR-05 Usability:** Mobile-responsive interface adhering to WCAG 2.1 accessibility standards with intuitive dark/light theme switching.

3. System Architecture & Dual-Store Resilience

The system employs a decomposed multi-tier architecture dividing responsibilities between the Presentation Layer, the Application Gateway, and a Resilient Persistence Subsystem.

```
Architectural Flow Overview:
[Client Browser: React 18 SPA] <--- HTTPS / WSS ---> [Express API Gateway & Socket.IO Bus]
|
+-------------------------------+-------------------------------+
| |
[Primary Database: MongoDB Atlas] [Fallback Store: Local JSON Engine]
- Cloud Replica Set (M0/M10) - Structured JSON Collections
- Mongoose Document ODM - Atomic FS Rename Writes
- High-concurrency indexing - Zero Cloud Dependency
```

3.1 Dual-Store Failover & Reconnect Algorithm

To insulate academic operations against internet outages or cloud database connection drops, the backend implements an autonomous persistence abstraction layer (`server/store/dualStore.ts`).

- **Heartbeat Monitor:** A lightweight background timer regularly pings MongoDB Atlas with a 2000ms timeout threshold.
- **Autonomous Degraded Mode:** If MongoDB pings fail, the active driver automatically switches internal write/read pointers to local JSON files stored under `server/data/`.
- **Atomic Disk Commits:** Local updates are written first to temporary buffer files and committed using atomic file replacement (`fs.renameSync`), guaranteeing zero file corruption during power failures.
- **Transparent DTO Contract:** Client endpoints receive identical response payloads regardless of active store, accompanied by diagnostic header `X-Data-Source: mongo | json`.
- **Self-Healing Reconnection:** When Atlas connectivity is re-established, the backend triggers reconciliation routines synchronizing local mutations back to cloud collections.

4. Core Business Algorithms & Engines

4.1 Timetable Conflict Detection Algorithm

When an administrator schedules a class period (`POST /api/timetable/class`), the system validates three mutually exclusive constraints before persisting the record:

```
Algorithm: ValidateTimetablePeriod(NewPeriod P)
1. Let R = P.room, F = P.facultyId, C = {P.department, P.semester, P.section}, T = {P.day, P.periodSlot}
2. Query existing periods matching slot T:
a. Room Clash Check: If any period has period.room == R, throw 'Classroom R is already reserved for cohort ' + period.section
b. Faculty Clash Check: If any period has period.facultyId == F, throw 'Faculty member is already scheduled in room ' + period.room
c. Cohort Clash Check: If any period matches cohort C, throw 'Section ' + C.section + ' already has a scheduled class during this period'
3. If all checks evaluate false, commit P to database.
4. Broadcast event timetable_updated with payload { department, semester, section } via Socket.IO.
```

4.2 Anti-Tamper Attendance Single-Day Enforcement

Attendance fraud frequently occurs when instructors backdate records. Edu Track resolves this at both UI and API layers: The date picker is strictly locked to the current system date (`new Date().toISOString().split('T')[0]`). On server receipt, the backend verifies that the submitted session date matches the server's current calendar day (within a 12-hour timezone tolerance), rejecting historic or future timestamps.

4.3 Online Exam Lifecycle & Proctoring State Machine

Online exams progress through an immutable state machine:

- **DRAFT:** Exam created by instructor; questions and timings editable.
- **SCHEDULED:** Exam published; students see start date and preparation guidelines.

- **ACTIVE (AVAILABLE):** Active examination window. Single-attempt rule enforced. Countdown timer active. Tab switches monitored via browser `window.onblur` and `document.visibilitychange`.
- **CLOSED:** Examination deadline passed; no new attempts permitted.
- **RESULTS RELEASED:** Faculty clicks 'Release Results'. Written attempts unlock student scorecards, percentage calculation, pass/fail status, and question answer keys.

5. Verification, Testing & Quality Assurance

The platform underwent systematic test procedures covering unit components, integration bridges, network fault injection, and user role boundaries.

Test Case ID	Test Scope / Condition	Expected Behavior	Result
TC-SEC-01	Access admin routes with student JWT token	HTTP 403 Forbidden returned; unauthorized route blocked.	**PASSED**
TC-TIME-01	Schedule 2 classes in Room 301 at Monday 10:00 AM	Collision engine intercepts conflict; outputs descriptive alert.	**PASSED**
TC-ATT-01	Attempt to submit attendance for yesterday's date	Date picker locked to today; API rejects out-of-bounds date.	**PASSED**
TC-FAIL-01	Simulate MongoDB network disconnection	Store switches to local JSON; write succeeds with zero errors.	**PASSED**
TC-EXAM-01	Student completes exam; attempts second submission	System flags attempt as already submitted; access barred.	**PASSED**
TC-EXAM-02	Student switches browser tabs during exam	Proctoring alert modal displays; tab switch count logged.	**PASSED**
TC-EXAM-03	Faculty clicks 'Release Results' for closed exam	Student portal reveals scores and question review solutions.	**PASSED**

6. Repository Structure & Deployment Topology

To satisfy industry modularity standards and simplify CI/CD pipelines, the codebase is arranged in a clean tripartite layout:

```
Edu-Track-Student-Academic-Management-System/
■■■ frontend/ # React 18 + Vite + TypeScript frontend source code
■ ■■■ src/ # Components, portals, hooks, and services
■ ■■■ public/ # Static assets, branding, and icons
■ ■■■ package.json # Frontend dependencies & Vite scripts
■ ■■■ vite.config.ts # Optimized bundler configuration
■■■ backend/ # Node.js + Express backend service
■ ■■■ routes/ # Modular REST endpoints (auth, timetable, exam, etc.)
■ ■■■ store/ # DualStore engine & JSON fallback persistence
■ ■■■ data/ # Atomic local JSON database files
■ ■■■ server.ts # Gateway entrypoint & Socket.IO initialization
■ ■■■ package.json # Server dependencies & scripts
■■■ docs/ # Official project engineering documents
■■■ Project_PPT.pdf
■■■ Project_Documentation.pdf
■■■ Database_Design.pdf
■■■ API_Documentation.pdf
```

6.1 Production Hosting Configuration

- **Frontend Hosting:** Deployed on Vercel Edge Network with automatic Brotli/Gzip compression, HTTP/2 multiplexing, and SPA client routing rewrites configured via `vercel.json`.
- **Backend Hosting:** Node.js process managed via PM2 on scalable cloud compute, binding to port 5000 with CORS authorization allowing the live Vercel domain.
- **Database Hosting:** MongoDB Atlas cloud replica cluster with automated volume backups and TLS encryption in transit.

7. Conclusion & Future Roadmap

Edu Track demonstrates how modern full-stack web engineering solves longstanding university administrative challenges. By enforcing strict role-based access control, validating timetable collisions proactively, preventing attendance backdating, providing a secure online examination environment, and guaranteeing high availability via dual-store resilience, the system delivers a dependable, scalable, and intuitive digital foundation for academic institutions.

Planned enhancements include cross-platform mobile apps using React Native, automated AI student performance diagnostics, biometric attendance integration, and automated institutional accreditation report generation.